402Lab — video scripts (Colosseum Crypto World's Fair)

Two videos, both required by Colosseum:

Video	Length	Question it answers	Format
Pitch	**max 3:00**	*Why* — team, problem, user, vision	Voiceover over slides (+ webcam bubble)
Technical demo	**2:00–3:00**	*How* — what was built, Solana integration, decisions	Screen recording with voiceover

Judges watch the pitch first and decide from it whether to look deeper. A clear narrative beats editing: slides + voice is enough. Do not exceed 3:00 and do not turn the demo into a second pitch.

Speech is in English (~140 words per minute). Lines in `[brackets]` must be filled with real facts only — cut them rather than invent.

---

1. Pitch video (target 2:50)

0:00–0:15 · Hook — slide: terminal line `402 Payment Required`

> AI agents can already write code, research and plan trips. But the moment an agent needs a paid API — OCR, translation, an LLM — it hits a wall built for humans: sign up, type a card, buy prepaid credits, copy an API key. Once per provider.

0:15–0:35 · Team — slide: photo, name, background

> I'm Emerson Silva, tech lead at Porto Seguro in São Paulo. For ten years I've built web interfaces for money — including Santander's FX portal on Ripple, which moved cross-border transfers from days to minutes. Today I'm focused on AI: I'm finishing a postgraduate degree in Applied AI Engineering. I built 402Lab during the Crypto World's Fair.

0:35–1:00 · Problem — slide: three stat cards

> Cards can't charge cents: 2.9% plus 30 cents on a one-cent API call is a **3,003%** fee. Prepaid credit is locked per vendor — OpenAI asks five dollars minimum, and it expires in a year. In Brazil it's worse: **3.5% IOF** on every international card purchase, and a quarter of the population has no credit card at all. And half of API teams say they fear agents making excessive or unauthorized calls.

1:00–1:40 · Solution — slide: allowance card + flow (agent → 402 → USDC → split)

> 402Lab is a spending account for AI agents. The human gives the agent an **allowance**: a cap per call and a cap per day. The agent pays each API call in USDC on Solana through HTTP 402 — the x402 protocol. No popup, no SOL for gas, settled in about a second. Providers get paid automatically: every 30 seconds one Solana transaction splits the revenue — 85% to the provider, 5% to the integrator, 10% to the platform. And the human watches every payment live, on the web and on their phone, pauses the agent with one tap, and gets one statement at the end of the month.

1:40–2:05 · Proof — 20 s clip cut from the technical demo

> Here Claude, through our MCP server, reads an image, translates it and summarizes it: three providers, three payments, 1.7 cents in total. By card plus IOF, that's 95 cents. I tap Pause on the phone — and the agent's next call is refused before any money moves.

2:05–2:30 · Why us, why Solana, business — slide: "x402 is the rail. 402Lab is the account."

> Generic x402 gateways already exist. What's missing is the **account**: limits a human sets, an agent cannot exceed, and a statement at the end of the month. Solana is what makes it viable — sub-cent fees, fast finality and native USDC. We earn the 10% platform share of every call, and a provider joins by answering 402 with a price.

2:30–2:50 · Validation + vision — slide: roadmap

> [Real validation only: "I showed it to N developers / API providers and they said ...". Cut if none.] Next: mainnet with a licensed custody partner, Pix on-ramp for Brazilians without a card, and an on-chain splitter program. 402Lab — pocket money for AI agents.

Slides (7)

1. `402 Payment Required` in mono, lime on black — 402Lab logo in the corner
2. Team — photo + one line
3. Problem — 3,003% · 3.5% IOF · $5 minimum that expires
4. Solution — allowance card (`$0.02/call · $1/day`) + four-step flow
5. (video clip)
6. "x402 is the rail. 402Lab is the account." + business model (10% of each call)
7. Roadmap + links: live demo, GitHub, logo

---

2. Technical demo video (target 2:45)

Record each block as a separate take, then join them. Browser at 125% zoom, dark mode, Do Not Disturb on, bookmarks bar hidden.

0:00–0:20 · Architecture — README architecture diagram (or slide)

> 402Lab is an npm workspaces monorepo in TypeScript. A Fastify API with `@x402/fastify` sells ten paid services from nine providers and keeps a Postgres ledger. A Next.js dashboard, an Expo companion app, and an MCP server that lets any agent pay. Solana devnet with `@solana/kit`, USDC as SPL token, settlement through the x402.org facilitator.

0:20–0:45 · The 402 — terminal

```
curl -i -X POST -H 'content-type: application/json' -d '{}' \
  https://api-production-570e4.up.railway.app/svc/translate
```

> Every paid route answers **402** with a `payment-required` header: scheme `exact`, network Solana devnet, the USDC mint, our treasury as `payTo`, the facilitator as fee payer, and the price in atomic units — 2000, that's two-tenths of a cent. Money is a bigint end to end; it never touches a float.

0:45–1:25 · The agent pays — Claude Code + dashboard side by side

Prompt in Claude: *"Extract the text from this image, translate it to English and summarize it."*

> Claude calls `list_services`, then `call_service` three times. The MCP server holds a session keypair: it gets the 402, signs a `TransferChecked` and retries — no popup, and the facilitator pays the network fee, so the agent never needs SOL. On the right, each payment lands on the dashboard live, with its signature. [Click one → Solscan shows the real devnet transaction.]

1:25–1:50 · The policy — phone + `apps/api/src/features/ledger/hooks.ts`

Tap **Pause** on the phone, ask Claude for one more call.

> The allowance policy — paused, per-call cap, daily cap — runs in the x402 `onAfterVerify` hook, after the signature is verified and **before** settle. A refusal aborts the payment, so it never reaches the facilitator and no USDC moves. Daily spend is reserved inside a transaction, so two parallel calls can't both slip under the cap.

1:50–2:20 · The split — `/app/splits` + Solscan + `apps/api/src/features/splits/worker.ts`

> x402's `exact` scheme pays a single recipient, and the splits extension is still an open PR. So payments land in the treasury and a worker settles the split every 30 seconds: it sums the payouts per recipient and sends **one** Solana transaction with up to eight `TransferChecked` instructions. Provider and integrator are floored, the remainder goes to the platform, so value is always conserved. The signature is saved before sending, and a reconciler checks it against the last valid block height — a crash can't pay twice or lose a payout.

2:20–2:45 · Decisions and what's next — slide or README

> What we chose not to build: mainnet waits for licensed custody under Brazil's new central-bank rules, and Pix and invoices are simulated and labelled. The API contract is OpenAPI-checked in CI, and every merge to `main` deploys to Railway and smoke-tests production. Next: an Anchor splitter program so the split becomes atomic with the payment.

Recording checklist

- [ ] Devnet funded and `npm run e2e:devnet` green — real Solscan links for the demo
- [ ] Session wallet funded, allowance created, MCP added to Claude (`claude mcp add 402lab -e API_URL=https://api-production-570e4.up.railway.app -- node "$PWD/packages/mcp/dist/cli.js"`)
- [ ] Phone with the app paired to the allowance
- [ ] Record: macOS `Cmd+Shift+5` (or Loom / Screen Studio), 1080p, mic close to mouth
- [ ] Edit: iMovie or CapCut — cut pauses, add the logo on the first and last frame
- [ ] Upload: YouTube **unlisted** (or Loom), open the link in a private window to confirm access
- [ ] Both under the time limit (pitch ≤ 3:00, demo 2:00–3:00)

3. Technical demo — mock version (no devnet funds needed)

Use this when the devnet wallets are not funded. Everything shown is honest: the 402 comes from the real production API, the UI runs on the demo build with its amber "Mock data" banner, and the on-chain parts are shown in code. Judges can open the same link and see the same thing.

Tabs to open before recording:

1. Terminal with the `curl` below
2. https://web-demo-production-0d9a.up.railway.app/app (click **Connect wallet**: the demo wallet connects without an extension)
3. VS Code with `apps/api/src/features/ledger/hooks.ts` and `apps/api/src/features/splits/worker.ts`
4. Optional: the phone with the demo APK, **Try the demo**, opened on `research-bot`

0:00–0:20 · Architecture — same as section 2

0:20–0:45 · The real 402 — terminal

```
curl -i -X POST -H 'content-type: application/json' -d '{}' \
  https://api-production-570e4.up.railway.app/svc/translate
```

Same narration as section 2. Add at the end:

> That 402 is our live production API. For the rest of the walkthrough I'm on the demo build — it runs the same UI on simulated data, labelled on screen, so anyone can open it without devnet USDC.

0:45–1:15 · Allowance and live payments — web demo

Click **Allowances → research-bot**. Wait for two or three rows to slide into *Live payments* (one every 4 s). Then **Playground**: run OCR, Translate, Summarize.

> Each agent gets an allowance: here two cents per call and one dollar per day. Payments stream in live over server-sent events, each with its service, provider, amount and Solscan link. In the playground I call three paid services — each returns its result plus the x402 payment response. The real path is the same through our MCP server: Claude gets the 402, the session keypair signs a `TransferChecked`, the facilitator pays the fee.

1:15–1:45 · Pause and the policy — web (or phone) + `hooks.ts`

Click **Pause agent**: the pill turns PAUSED. Cut to `hooks.ts`, highlight the `abort` lines.

> One tap pauses the agent. On the server the policy — paused, per-call cap, daily cap — runs in the x402 `onAfterVerify` hook, after the signature is verified and before settle. A refusal aborts the payment: it never reaches the facilitator, so no USDC moves. Even a throwing hook aborts, because x402 would otherwise let the payment through.

1:45–2:15 · Split — `/app/splits` + `worker.ts`

> x402's `exact` scheme pays one recipient, so payments land in the treasury and a worker splits them every 30 seconds: payouts summed per recipient, one Solana transaction with up to eight `TransferChecked`. Provider and integrator are floored, the platform keeps the remainder, so value is conserved. The signature is saved before sending and reconciled against the last valid block height — a crash can't pay twice.

2:15–2:45 · Statement + decisions — `/app/statement`

> At the end of the month the human gets one statement per agent, with the card-plus-IOF comparison. Mainnet waits for licensed custody; Pix and invoices are simulated. `npm run e2e:devnet` runs this whole flow against devnet, and every merge to `main` deploys and smoke-tests production.